

day 5

JavaScript Advanced Guide: DOM, Form Validation & BOM

📁 أولاً: ملف الـ DOM (Document Object Model)

الـ DOM هو بنية شجرية (Tree Structure) يمثل بها المتصفح ملف الـ HTML ليسمح للجافا سكريبت بالوصول للعناصر وتعديلها.

1. الـ DOM Traversal (التنقل بين العناصر)

التنقل يعتمد على علاقة الأبوة والأخوة بين الـ Tags (العناصر):

- `children`: معين Tag بكل العناصر الأبناء المباشرين لـ (HTMLCollection) ترجع مصفوفة.
- `firstElementChild`: داخل العنصر (Element) للوصول لأول ابن مباشر.

```
// document وهو الابن الأول للـ <html> الوصول لـ
console.log(document.children[0]);

// (أندكس 0) <html> وهو أول ابن داخل <head> الوصول لـ
console.log(document.children[0].children[0]);

// (أندكس 1) <html> وهو ثاني ابن داخل <body> الوصول لـ
console.log(document.children[0].children[1]);

// children دون الحاجة لكتابة <body> اختصار سريع ومباشر للوصول لـ
console.log(document.body);

// أول عنصر ابن -> أول عنصر ابن داخله -> body الانتقال: الدخول لـ
var element = document.body.firstElementChild.children[0];
console.log(element);
```

2. طرق اختيار العناصر (Selection Methods)

هناك 5 طرق رئيسية لاستهداف عناصر الـ HTML من الـ DOM:

الطريقة	المخرجات	ملحوظة مهمة
<code>getElementsByTagName("tag")</code>	شبه HTMLCollection (مصفوفة)	نستخدم <code>[0]</code> لاستهداف أول عنصر بالتحديد.
<code>getElementsByClassName("class")</code>	شبه HTMLCollection (مصفوفة)	تستهدف العناصر بناءً على الكلاس.
<code>getElementById("id")</code>	عنصر واحد فقط	لأن ID متكرر بالخطأ في HTML، هتارجع أول واحد بس وتطنش الباقي.
<code>querySelector("selector")</code>	أول عنصر مطابق للـ CSS Selector	مرنة جداً وتأخذ كلاس . أو أيدي # أو تاغ مباشر.
<code>querySelectorAll("selector")</code>	شبه مصفوفة NodeList	ترجع جميع العناصر المطابقة للشرط.

أمثلة الكود:

JavaScript

```
// 1. باستهداف التاغ div
var myDivTag = document.getElementsByTagName("div")[0];
console.log(myDivTag);

// 2. باستهداف الكلاس last-div
var myDivClass = document.getElementsByClassName("last-div")[0];
console.log(myDivClass);

// 3. باستهداف الأيدي id-para
var myParagraphID = document.getElementById("id-para");
console.log(myParagraphID);

// 4. يقابله span استهداف أول
var mySpan = document.querySelector("span");
console.log(mySpan);

// 5. NodeList في الصفحة كـ spans استهداف جميع الـ
var myAllSpans = document.querySelectorAll("span");
console.log(myAllSpans);

// 💡 CSS Selector تركة الـ querySelector:
// (span) موجود داخل عنصر (a) استهداف أول رابط
console.log(document.querySelector("span a"));
```

3. التحكم في الخصائص (Working with Attributes)

لتعديل خصائص الـ HTML (مثل href, src, target) لدينا طريقتان:

1. الدوال الرسمية: `()setAttribute` و `()getAttribute`.
2. الوصول المباشر للخاصية (Direct property access) عبر النقطة . .

JavaScript

```
var myAnchorID = document.getElementById("my-a");
console.log(myAnchorID);

// إضافة أو تعديل خاصية الرابط والهدف لفتح صفحة جديدة
myAnchorID.setAttribute("href", "[https://www.youtube.com/](https://www.youtube.com/)");
myAnchorID.setAttribute("target", "_blank");

// تعديل مباشر بالخاصية (Direct Access)
myAnchorID.href = "[https://www.facebook.com/](https://www.facebook.com/)";

// قراءة قيمة الخاصية الحالية
console.log(myAnchorID.getAttribute("href")); // يرجع القيمة المكتوبة نصاً
// بالظبط
console.log(myAnchorID.href); // يرجع الـ Full URL
// المتصفح
```

4. التنسيق وتغيير الـ Styles

نغير شكل العناصر بأسلوبين:

1. **تعديل الكلاسات (classList):** لإضافة كلاس مكتوب مسبقاً في الـ CSS (مثل كلاس `red-class` الموجود بملف الـ HTML).
2. **الـ Inline Style المباشر:** يكتب التنسيق مباشرة داخل الـ Tag في الـ HTML.

JavaScript

```
var myParagraphID = document.getElementById("id-para");

// أ) إضافة كلاس جاهز للعنصر
myParagraphID.classList.add("red-class");
// myParagraphID.classList.remove("red-class"); // لحذف الكلاس

// تتحول CSS تذكر: الخصائص التي فيها شرطة بالـ (⚠ Inline Style ب) إضافة الـ (بجافاسكريبت CamelCase لـ
myParagraphID.style.backgroundColor = "yellow"; // background-color بدلاً من
myParagraphID.style.color = "black";
myParagraphID.style.fontSize = "22px";

// Inline تنبيه: ده بيمسح الـ (💡 attribute ج) تعديل كل الستاين مرة واحدة كـ //
```

```
(القديم ويكتب الجديد مكانه style  
myParagraphID.setAttribute("style", "color:lightGreen; background-color:  
purple");
```

5. إنشاء، نسخ، وإضافة العناصر للـ DOM

- `document.createElement("tag")` : ينشئ تاغ جديد تماماً في الذاكرة ولكنه لا يظهر في الصفحة حتى نقوم بعمل `append`.
- `cloneNode(deep)` : ينسخ العنصر بالكامل.
 - لو ممررنا `true` (Deep Clone) : ينسخ التاغ بجميع أولاده ونصوصه الداخلية.
 - لو ممررنا `false` : ينسخ التاغ الخارجي فقط وهو فارغ.

JavaScript

```
// في الذاكرة وتنسيقه h2 إنشاء عنصر 1.  
var myH2 = document.createElement("h2");  
myH2.style.backgroundColor = "green";  
myH2.style.color = "white";  
myH2.textContent = "Hello Iam H2";  
console.log(myH2);  
  
// 2. نسخ العنصر (تعديل النسخة لا يؤثر على الأصل)  
var myCopyElement = myH2.cloneNode(true);  
myCopyElement.style.backgroundColor = "black";  
console.log(myCopyElement);  
  
myH2.style.backgroundColor = "purple"; // تعديل لون الأصل بربل والنسخة تفضل بلاك  
كما هي  
  
// 3. (الظهور الفعلي بالصفحة) DOM الإضافة للـ  
document.body.append(myH2); // body يضاف في نهاية الـ  
  
// محدد بالأيدي div إلحاقه داخل  
var myDivAppend = document.getElementById("append-div");  
myDivAppend.append(myH2); // ملحوظة: العنصر لا يتكرر، بل ينتقل من مكانه ⚠️  
!القديم للمكان الجديد  
  
// 4. مع الأبناء Cloning تركة الـ  
var myDivID = document.getElementById("div-id");  
var myDeepCopy = myDivID.cloneNode(true); // (p) بالـ 3 فقرات div ينسخ الـ  
اللي جواه  
var myShallowCopy = myDivID.cloneNode(false); // الفاضي برة بس div ينسخ الـ  
وطنش اللي جواه
```

6. التعامل مع الأحداث (Event Handling)

ربط الدوال بالـ HTML تفاعلياً:

- الكلمة المحجوزة `this` في الـ HTML تعني "مرر هذا العنصر الحالي بنفسه للدالة كـ **Parameter**".

JavaScript

```
// دالة تغيير لون الخلفية لعنصر محدد يتم تمريره لها
function changeColor(div) {
    div.style.backgroundColor = "green";
}

// دالة تعرض النص الداخلي للعنصر الممرر في Alert
function alertData(x) {
    alert(x.textContent); // بناء على الزرار المكبوس
}
```



ثانياً: ملف التحقق والوقت (Form Validation & Timers)

الملف ده بيتحكم في التحقق من صحة الفورم (Validation) وبيمنع إرسال البيانات الخاطئة، وبيتعامل مع توقيتات المتصفح.

1. الـ Form Validation والـ Query String

- وبيعت البيانات Submit تمنع الفورم إنه يعمل) دالة توقف السلوك الافتراضي للمتصفح فوراً : `e.preventDefault()` (للسفحة الثانية لو فيه غلط).

JavaScript

```
var inp = document.querySelector("input"); // في الفورم input استهداف أول
var errorSpan = document.getElementById("validateError"); // مكان عرض رسالة الإيـرور

// Submit أ) دالة تفحص البيانات عند ضغط زر الـ
function validationSubmit(e) {
    if (inp.value === "") { // لو الحقل فارغ
        e.preventDefault(); // امنع الإرسال
        errorSpan.textContent = "you should fill this input";
    }
    else if (isFinite(inp.value)) { // لأن الحقل مخصص للحروف
        // لو كتب أرقام بس (لأن الحقل مخصص للحروف)
        // الاسم
    }
}
```

```

        e.preventDefault(); // ا منع الإرسال
        errorSpan.textContent = "you should enter characters only";
    }
}

// (onchange) ب) دالة تفحص البيانات مباشرة أثناء كتابة المستخدم
function inputValidation() {
    if (isFinite(inp.value) && inp.value !== "") {
        errorSpan.textContent = "you should enter characters only";
    }
    else {
        errorSpan.textContent = ""; // نظف الإيـرور لو كاتب حروف صح
    }
}

```

🚀 تركة استقبال ومعالجة البيانات بالصفحة الثانية (welcome.html):

الـفورم لو اتبعت بنجاح، المتصفح ينقلنا لصفحة welcome.html ويبط البيانات في الرابط فوق على هيئة Query String بتبدأ بعلامة استفهام ? . firstName=ali&lastName=salem كود الصفحة الثانية بيـفك الشفرة دي كالتالي:

JavaScript

```

var userDate = location.search; // بيـجيب النم "?firstName=ali&lastName=salem"
console.log(userDate);

var userDataArray = userDate.split("&"); // فيطلع مصفوفة
من قطعتين
var firstName = userDataArray[0].split("=")[1]; // بياخد القطعة الأولى ويقطعها
"ali" عند ال = وياخد القيمة
console.log(firstName);

var lastName = userDataArray[1].split("=")[1]; // بياخد القطعة الثانية ويقطعها
"salem" عند ال = وياخد القيمة
console.log(lastName);

// طباعة رسالة الترحيب باسم المستخدم ديناميكياً في الصفحة الثانية
document.write('<h1>welcome ${firstName} ${lastName} </h1>');

```

2. التوقيـتات (Timers: setTimeout & setInterval)

- **setTimeout** : (ثانية = ms بملي ثانية، 1000) تنفذ الكود مرة واحدة فقط بعد مرور وقت محدد.
- **setInterval** : تكرر تنفيذ الكود بشكل مستمر ودوري كل فترة زمنية محددة. لإيقافها، يجب تخزينها في متغير وتـمـريـره : **clearInterval()** .

JavaScript

```
// تنفيذ الكود مرة واحدة بعد 5 ثوان (5000ms)
setTimeout(function () {
    console.log("hello");
}, 5000);

// بشكل لانهائي (500ms) تكرار الكود كل نصف ثانية
var clearIntervalVariable = setInterval(function () {
    console.log("hello");
}, 500);

// HTML مرتبطة بزرار في الـ (اللانهائي عند استدعائها Interval دالة توقف الـ)
function clearInt() {
    clearInterval(clearIntervalVariable);
}
```



```
// implement each time
var myInterval = setInterval(function () {
    console.log("hello")
}, 2000)

function clearInt(){
    clearInterval(myInterval)
}
```

ثالثاً: ملف الـ BOM (Browser Object Model)

الـ BOM يتعامل مع كل حاجة برة صفحة الـ HTML وتخص نافذة المتصفح نفسه (الـ Window).

JavaScript

```
var child; // متغير عام لتخزين مرجع النافذة الجديدة (الابن)

// يعرض صفحة معينة (Pop-up Tab) فتح نافذة أو لسان جديد 1.
function openWind() {
    child = window.open("page2.html"); // يفتح الصفحة ويخزنها جوه المتغير
    child
}

// 2. إغلاق النافذة المفتوحة
function closeWind() {
    child.close(); // ⚠️ تنبيه: المتصفحات تمنع إغلاق النافذة بـ
    // open() كانت جافا سكريبت هي اللي فتحتها بـ
}

// 3. فتح نافذة الطابعة الخاصة بالمتصفح لطباعة الصفحة الحالية
```

```

function printWind() {
    window.print();
}

// 4. التحكم في سجل التنقل الخاص بالمتصفح (History)
function historyTab() {
    // history.back() // (مثل زر الخلف في المتصفح) للرجوع خطوة للخلف
    // history.forward() // للتقدم خطوة للأمام
    history.go(-1); // دالة مرنة: مرري لها 1 للرجوع خطوة، أو 2- أو 1 للتقدم خطوة
    // لخطوتين، أو 1 للتقدم خطوة
}

// 5. التحكم في رابط الصفحة الحالي (Location)
function replacePage() {
    // بتحول المستخدم للموقع الجديد، و ⚠ تركتها: بتمسح الصفحة الحالية من الـ
    History
    // في المتصفح مش هيرجع لصفحتنا Back فالمستخدم لو دخل جوجل وداس على زرار
    // ثاني!
    location.replace("[http://www.google.com](http://www.google.com)");
}

```

⚠ الفرق بين الـ `html collection` و `odelist`

`html collection` >> html elements only

`odelist` >> html elements ,text ,comment ,anything